DOCUMENTO TÉCNICO · VERSIÓN 1.0

Sistema de Gestión Hospitalaria

Proyecto-DW

Alcance final, arquitectura, tecnologías y estado de avance

Documento de síntesis para presentación del proyecto

Aplicación web para la gestión operativa de una red hospitalaria: estructura organizativa, personal médico, pacientes, turnos, consultas, informes y reportes de gestión.

Autor	Jose Ignacio Bravo
Fecha	14 de noviembre de 2026
Repositorio	Proyecto-DW (Git)
Stack principal	Spring Boot · PostgreSQL · HTML/CSS/JavaScript

Documento generado a partir del código fuente, la configuración y la base de datos del repositorio al 14/11/2026.

Ficha del documento

Campo	Detalle
Proyecto	Proyecto-DW — Sistema de Gestión Hospitalaria
Tipo de solución	Aplicación web con API REST y base de datos relacional
Objetivo del documento	Resumir el alcance final, la arquitectura, las tecnologías y el avance real del proyecto
Autor	Jose Ignacio Bravo
Fecha de corte	14 de noviembre de 2026
Versión	1.0
Estado general	MVP en construcción: interfaz y autenticación operativas; integración de datos y CRUD en proceso
Fuentes	Código fuente del repositorio, configuración de Spring Boot, scripts de despliegue y base de datos PostgreSQL local

Contenido

1. Resumen ejecutivo
2. Introducción y objetivos
3. Alcance del proyecto
4. Arquitectura del sistema
5. Tecnologías utilizadas
6. Modelo de datos
7. Backend y API REST
8. Frontend y experiencia de usuario
9. Estado de avance
10. Despliegue y ejecución local
11. Observaciones técnicas y riesgos
12. Próximos pasos
13. Conclusiones

Anexo A. Funciones de base de datos

Anexo B. Glosario

1. Resumen ejecutivo

Proyecto-DW es una aplicación web orientada a la gestión operativa de una red hospitalaria. Su propósito es centralizar en un solo sistema la estructura organizativa de los hospitales (hospitales, departamentos y unidades), el personal médico, los pacientes y su admisión en unidades, los turnos de atención, las consultas y los informes, además de generar nueve reportes de gestión que apoyan la toma de decisiones.

La solución se apoya en una arquitectura web en tres capas. El frontend está construido con HTML5, CSS3 y JavaScript moderno sin framework de build; el backend es una API REST desarrollada con Spring Boot 3.2.5 sobre Java; y la persistencia se resuelve con PostgreSQL 17, donde una parte importante de las reglas de negocio se implementa mediante funciones PL/pgSQL y disparadores. Spring Boot sirve, en el mismo puerto, tanto los archivos estáticos del frontend como los endpoints de la API.

A la fecha de este documento el proyecto cuenta con una base sólida: el modelo de datos y la lógica SQL están construidos y cargados con datos de prueba; la autenticación con sesiones, BCrypt y roles ADMIN/MEDICO está operativa; el panel de control presenta una interfaz responsive con los módulos Resumen, Turnos y Pacientes; y los nueve reportes cuentan con endpoints y funciones de base de datos. El proyecto se encuentra en fase de integración: falta conectar el panel a los datos reales de forma completa, exponer los CRUD del dominio por REST, construir la vista de reportes y versionar la base de datos dentro del repositorio para que el despliegue sea reproducible.

> **Idea fuerza**
>
> El valor del proyecto ya está demostrado en dos frentes: un modelo de datos hospitalario completo con reglas de negocio automatizadas, y una interfaz de usuario profesional y funcional. El siguiente salto no es de diseño, sino de integración: conectar esa interfaz con los servicios y datos que ya existen.

2. Introducción y objetivos

2.1 Contexto y problema

La gestión de una red hospitalaria implica manejar información heterogénea y altamente relacionada: hospitales, departamentos, unidades, médicos, pacientes, turnos, consultas e informes. Cuando esa información se lleva en hojas de cálculo o registros aislados, aparecen duplicidades, pérdida de trazabilidad y dificultades para obtener indicadores confiables. El proyecto surge para resolver esa fragmentación con una aplicación web única, accesible desde un navegador y con reglas de negocio centralizadas.

El sistema está pensado para dos perfiles: administradores, que configuran la estructura y consultan los reportes globales; y médicos, que trabajan con la operación diaria de turnos y pacientes. La interfaz se adapta a escritorio, tablet y móvil, de modo que pueda usarse en distintos puestos de trabajo dentro del hospital.

2.2 Objetivo general

Desarrollar una aplicación web que centralice la gestión hospitalaria y permita administrar la estructura organizativa, el personal médico, los pacientes, los turnos, las consultas y los informes, con control de acceso por roles y reportes que faciliten el seguimiento de la operación.

2.3 Objetivos específicos

- **Modelar la estructura hospitalaria:** hospitales, departamentos y unidades, con integridad referencial y borrado en cascada.
- **Gestionar el personal médico:** código, nombre, especialidad, licencia, teléfono, experiencia, datos de contacto y unidad asignada.
- **Gestionar pacientes y admisiones:** historia clínica, datos personales, fecha de nacimiento, estado y unidad de registro.
- **Administrar turnos y consultas:** cantidad de pacientes, estado del turno, médico responsable y registro de atenciones.
- **Automatizar los informes:** recalcular por turno los pacientes atendidos, altas, admitidos y totales.
- **Generar nueve reportes de gestión:** listados, resúmenes por hospital, proceso de atención, pacientes no atendidos, consultas exitosas y unidades a revisar.
- **Incorporar autenticación y perfiles:** registro, inicio de sesión, cierre de sesión, edición de perfil y roles ADMIN/MEDICO.
- **Ofrecer una interfaz responsive:** panel con navegación por secciones, filtros, búsqueda y modales de carga.

3. Alcance del proyecto

3.1 Alcance funcional

El alcance final se organiza en ocho módulos funcionales. La tabla distingue entre lo que está implementado en código, lo que existe a nivel de servicios o base de datos y lo que aún está pendiente de integración.

Módulo	Qué resuelve	Estado actual
Autenticación y usuarios	Registro en dos pasos, inicio y cierre de sesión, consulta y edición del perfil, roles ADMIN/MEDICO, contraseñas con BCrypt.	**Implementado**
Estructura hospitalaria	Alta, modificación y baja de hospitales, departamentos y unidades; relación jerárquica y borrado en cascada.	Servicios y BD listos; falta API/UI
Profesionales	Gestión de médicos, especialidades, licencias, experiencia y unidad; límite de 10 médicos por unidad.	Servicios y trigger; falta API/UI
Pacientes y admisión	Directorio de pacientes, historia clínica, datos personales, fecha de nacimiento y registro en unidades.	Servicios y BD; UI con datos demo
Turnos	Programación de turnos, cantidad de pacientes, estado, unidad y médico; agenda y filtros.	UI demo y servicios; falta persistencia real

Módulo	Qué resuelve	Estado actual
Consultas e informes	Registro de atenciones por turno y cálculo automático del informe (atendidos, altas, admitidos, total).	BD y servicios; falta API/UI
Reportes	Nueve reportes de gestión: listados, resúmenes, proceso, no atendidos, consultas exitosas y unidades a revisar.	API implementada; falta vista en el panel
Configuración y ayuda	Preferencias generales del sistema y centro de ayuda con guías y preguntas frecuentes.	**Placeholder / pendiente**

3.2 Roles y permisos

Rol	Alcance	Restricciones
ADMIN	Acceso completo al panel: Resumen, Turnos, Pacientes, Profesionales y Reportes.	Ninguna dentro del sistema.
MEDICO	Acceso a Resumen, Turnos y Pacientes para la operación diaria.	No puede entrar a Profesionales ni a Reportes; el panel oculta esas opciones.

El primer usuario que se registra en el sistema queda automáticamente con rol ADMIN; los siguientes se registran como MEDICO. Un administrador puede promover usuarios desde la base de datos, aunque todavía no existe una pantalla específica de gestión de usuarios.

3.3 Reglas de negocio principales

- **Límite de médicos por unidad:** un trigger impide asignar más de 10 médicos a una misma unidad, tanto en alta como en modificación.
- **Umbral de éxito de un turno:** una unidad se considera exitosa cuando atiende al menos el 80 % de los pacientes del turno; por debajo de ese valor aparece en el reporte de unidades que deben revisar turnos.
- **Informe automático:** al insertar, modificar o eliminar una consulta, un trigger recalcula el informe del turno afectado (pacientes atendidos, altas, admitidos y total).
- **Edad mínima:** la tabla Paciente tiene una restricción CHECK sobre la fecha de nacimiento para exigir al menos 16 años.
- **Integridad referencial:** las relaciones entre hospitales, departamentos, unidades, médicos, pacientes, turnos, consultas e informes usan claves foráneas con borrado en cascada.
- **Validaciones de datos:** las funciones de alta y modificación verifican campos obligatorios, existencia de la unidad y duplicados antes de escribir.

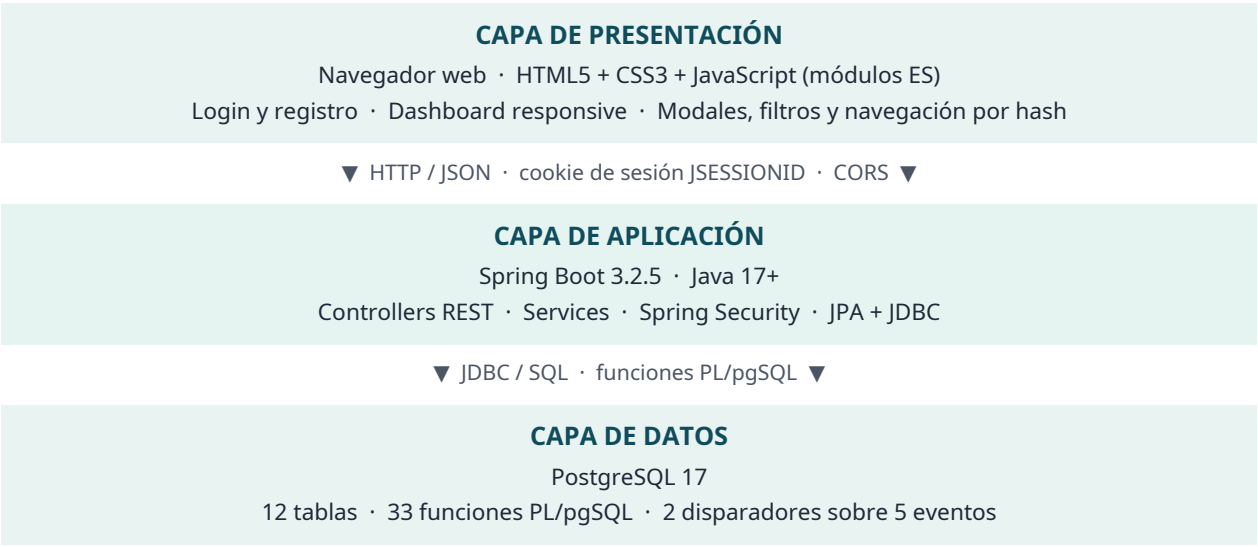
3.4 Límites del alcance

Esta versión no contempla facturación, farmacia, inventario, historia clínica completa con adjuntos, notificaciones automáticas, integración con sistemas externos ni aplicación móvil nativa. El foco está puesto en la gestión administrativa y operativa de la red hospitalaria y en los nueve reportes definidos.

4. Arquitectura del sistema

4.1 Visión general

El sistema sigue una arquitectura web en tres capas con despliegue monolítico. El navegador consume una interfaz estática servida por el propio Spring Boot; la interfaz se comunica por HTTP/JSON con los controladores REST; y los controladores delegan en servicios que acceden a PostgreSQL mediante JPA o JDBC. La lógica de negocio más compleja vive en funciones y disparadores de la base de datos.



4.2 Capa de presentación

Está compuesta por dos páginas HTML: index.html, que reúne la portada, el inicio de sesión y el registro en dos pasos; y dashboard.html, que contiene el panel principal con navegación por secciones. La lógica se separa en módulos ES: api.js centraliza las llamadas fetch y la detección automática de la URL del backend; auth.js gestiona login, registro y validaciones; dashboard.js administra el estado, el renderizado, los filtros y los modales. No hay framework ni proceso de build: el navegador carga los módulos directamente.

La interfaz es responsive, con barra lateral colapsable, menú móvil, tarjetas de indicadores, agendas, tablas, chips de estado, notificaciones tipo toast y modales para crear turnos, registrar pacientes y editar el perfil. El diseño usa una paleta propia, tipografías locales (Manrope y DM Sans) e iconografía de Bootstrap Icons y Lucide.

4.3 Capa de aplicación

Es una aplicación Spring Boot organizada por responsabilidades: config para seguridad y CORS; controller para los endpoints REST; service para la lógica de aplicación y el acceso a datos; model para las entidades; repository para las consultas JPA; y exception para el manejo global de errores. La autenticación se resuelve con Spring Security, sesiones HTTP y BCrypt. El dominio hospitalario se consulta con JDBC y funciones PL/pgSQL, mientras que los usuarios y perfiles se gestionan con Spring Data JPA.

4.4 Capa de datos

PostgreSQL 17 concentra el modelo relacional y una parte central de la lógica de negocio. El esquema hospitalario tiene nueve tablas (Hospital, Departamento, Unidad, Medico, Paciente, Registro, Turno, Consulta e Informe) y el esquema de usuarios tiene tres (user, data_user y reset_tokens). Sobre el dominio hay 33 funciones: 22 de alta, modificación y baja; 9 de reportes y consultas analíticas; y 2 de soporte para disparadores. Los disparadores automatizan el informe por turno y el límite de médicos por unidad.

4.5 Flujo de una petición

1. El usuario interactúa con el dashboard y se dispara una acción (por ejemplo, cargar el resumen o guardar un turno).
2. El módulo api.js construye la petición fetch con la URL del backend, el método HTTP y las cookies de sesión.
3. Spring Boot recibe la petición en el controlador correspondiente y valida la sesión cuando la operación lo requiere.
4. El controlador delega en un servicio, que ejecuta una consulta JDBC o una función PL/pgSQL en PostgreSQL.
5. La base de datos aplica validaciones, disparadores y reglas de negocio, y devuelve el resultado.
6. El servicio transforma el resultado en objetos o mapas, el controlador lo envuelve en JSON con el campo ok y el frontend actualiza la interfaz.

4.6 Organización del repositorio

```
Proyecto-DW/
├── backend/                  # API REST Spring Boot
│   ├── pom.xml
│   └── src/main/java/com/hospital/
│       ├── config/           # Seguridad y CORS
│       ├── controller/       # AuthController y ReportesController
│       ├── service/          # JPA, CRUD y REPORTES
│       ├── model/            # Entidades del dominio y de usuarios
│       ├── repository/       # UserRepository (Spring Data JPA)
│       └── exception/        # Manejo global de errores
├── frontend/                 # HTML + CSS + JavaScript (sin build)
│   ├── index.html            # Portada, login y registro
│   ├── dashboard.html        # Panel principal
│   ├── css/                  # Estilos del login y del dashboard
│   ├── js/                   # api.js, auth.js, dashboard.js
│   └── assets/               # Fuentes, imágenes e iconos
├── deploy/                   # Servicio systemd
├── docker-compose.yml        # Configuración parcial de contenedores
├── start.sh / stop.sh        # Arranque y parada local
└── estructura.txt            # Estructura objetivo del proyecto
```

5. Tecnologías utilizadas

La selección tecnológica prioriza simplicidad de despliegue, bajo acoplamiento y facilidad para ejecutar el sistema en un equipo de desarrollo sin infraestructura adicional.

Capa	Tecnología	Versión	Uso en el proyecto
Backend	Java	17 (compilación) / 25 (runtime actual)	Lenguaje de la API y de la lógica de aplicación.
Backend	Spring Boot	3.2.5	Arranque, configuración, servidor web y empaquetado.
Backend	Spring Web (MVC)	Incluido en Boot	Controladores REST y servicio de archivos estáticos.
Backend	Spring Data JPA + Hibernate	Incluido en Boot	Persistencia de usuarios y perfiles.
Backend	JDBC (DataSource)	Java SE	Acceso directo a funciones PL/pgSQL del dominio hospitalario.
Seguridad	Spring Security + BCrypt	Incluido en Boot	Hash de contraseñas, filtros de seguridad y CORS.
Datos	PostgreSQL	17	Base de datos relacional, funciones, disparadores y datos de prueba.
Datos	Driver PostgreSQL JDBC	Gestionado por Boot	Conexión entre Java y PostgreSQL.
Build	Maven	3.8.7	Dependencias, compilación y ejecución con spring-boot:run.
Frontend	HTML5 / CSS3	—	Estructura, diseño responsive y sistema visual.
Frontend	JavaScript ES Modules	ES2020+	Estado del panel, fetch, navegación y renderizado.
UI	Bootstrap Icons + Lucide	—	Iconografía de la interfaz.
UI	Manrope + DM Sans	Locales	Tipografías del sistema de diseño.
Versionado	Git + GitHub	—	Historial de cambios y respaldo del código.
Operación	Bash + systemd	—	Scripts de arranque/parada y servicio del sistema.
Contenedores	Docker Compose	—	Configuración parcial; todavía no operativa.

5.1 Criterios de selección

- **Spring Boot:** permite exponer la API y servir el frontend en un único proceso, lo que simplifica el despliegue local y la demostración del proyecto.
- **PostgreSQL:** ofrece integridad relacional, funciones y disparadores, adecuados para concentrar reglas de negocio como el límite de médicos o el cálculo automático de informes.
- **Frontend sin framework:** reduce dependencias y tiempos de build; el proyecto se puede abrir y modificar con herramientas básicas.
- **JPA + JDBC:** JPA simplifica el CRUD de usuarios; JDBC permite invocar funciones PL/pgSQL y mantener el control de las consultas analíticas.

6. Modelo de datos

6.1 Entidades y relaciones

El modelo hospitalario es jerárquico: un hospital contiene departamentos, un departamento contiene unidades y una unidad concentra médicos, turnos, informes y registros de pacientes. Los pacientes se vinculan a las unidades a través de la tabla Registro; las consultas relacionan un turno con un paciente; y los informes resumen la actividad de un turno en una unidad.

Tabla	Propósito	Campos principales	Registros
Hospital	Nodo superior de la estructura.	codHos, nomHos	8
Departamento	División de un hospital.	codDep, nomDep, codHos	23
Unidad	Área operativa de un departamento.	codUni, nomUni, ubiUni, codDep	26
Medico	Profesional asignado a una unidad.	codMed, nomMed, espMed, numLic, telMed, codUni, exp, datCon	23
Paciente	Datos personales del paciente.	numHisCli, nomPac, dirPac, fechaN	596
Registro	Admisión del paciente en una unidad.	estPac, codUni, numHisCli, fecha_admision	596
Turno	Turno de atención de una unidad.	numTur, cantAtenTur, estTur, codUni, codMed	26
Consulta	Atención de un paciente en un turno.	numTur, numHisCli, atendido, causa	18
Informe	Resumen automático de un turno.	numIn, hora, fecha, pacAten, pacAlt, cantAdm, total, codUni, numTur	47
user	Credenciales y rol del usuario.	username, email, password, role	2
data_user	Perfil extendido del usuario.	username, email, nombre, especialidad, direccion, telefono	2
reset_tokens	Soporte previsto para recuperación de contraseña.	id, email, token, fechas y estado	—

Los conteos corresponden a los datos de prueba cargados en la instancia local al momento de este documento. El modelo completo incluye claves primarias, claves foráneas con borrado en cascada y restricciones CHECK, como la edad mínima de 16 años para pacientes.

6.2 Lógica de negocio en la base de datos

Una decisión arquitectónica relevante es que las reglas críticas viven cerca del dato. Esto evita que distintos clientes o servicios puedan saltarse las validaciones y garantiza consistencia aunque la interfaz cambie.

Tipo	Cantidad	Función / disparador	Qué resuelve
CRUD	22	insertar / modificar / eliminar por entidad	Altas, cambios y bajas de hospitales, departamentos, unidades, médicos, pacientes, turnos, consultas e informes, con validaciones y mensajes de error.

Tipo	Cantidad	Función / disparador	Qué resuelve
Reportes	9	resumenHospitales, listadoMasDeCien, resumenProceso, listadoMedicos, listadoPacientes, pacientesNoAtendidos, informeDuranteConsultas, consultasExitosas, revisarTurnos	Consultas analíticas que alimentan los nueve reportes del sistema.
Soporte	2	actualizar_informe_trigger, limitarmedicosporunidad	Recalcula el informe al cambiar consultas e impide más de 10 médicos por unidad.

Además, hay ocho funciones específicas de usuarios en la base Login: creación, login, obtención, modificación de datos, cambio de contraseña y eliminación. En total, el sistema combina 41 funciones entre ambas bases.

6.3 Datos de prueba

La instancia local contiene un conjunto de datos representativo que permite validar la interfaz y los reportes: 8 hospitales, 23 departamentos, 26 unidades, 23 médicos, 596 pacientes con sus registros de admisión, 26 turnos, 18 consultas y 47 informes. También existen 2 usuarios de prueba, uno con rol ADMIN y otro con rol MEDICO.

7. Backend y API REST

7.1 Organización del código

El backend tiene 35 archivos Java y aproximadamente 3.600 líneas de código. La separación por paquetes facilita ubicar cada responsabilidad:

- **config:** SecurityConfig define la cadena de filtros, el encoder BCrypt y la configuración CORS; CorsConfig registra los orígenes permitidos.
- **controller:** AuthController expone login, registro, perfil y logout; ReportesController expone los nueve reportes.
- **service/JPA:** AuthService implementa autenticación, registro, perfiles y rol efectivo.
- **service/CRUD:** ocho servicios JDBC para hospitales, departamentos, unidades, médicos, pacientes, turnos, consultas e informes.
- **service/REPORTES:** seis servicios que invocan las funciones analíticas y arman las estructuras de respuesta.
- **model:** nueve entidades POJO del dominio y tres clases JPA para usuarios.
- **repository:** UserRepository con búsquedas por username/email y verificación de duplicados.
- **exception:** ApiExceptionHandler traduce errores de validación, base de datos y reportes a respuestas JSON uniformes.

7.2 API REST

La API se organiza en dos grupos: autenticación/usuarios y reportes. Todas las respuestas usan JSON y el campo booleano ok para indicar el resultado.

Método	Ruta	Descripción	Estado
POST	/api/login	Inicia sesión con usuario o email y contraseña; crea la sesión HTTP.	**Operativo**
POST	/api/register	Registra un usuario y su perfil; el primero queda como ADMIN.	**Operativo**
GET	/api/me	Devuelve el usuario de la sesión actual.	**Operativo**
PUT	/api/me	Actualiza nombre, especialidad, dirección y teléfono del perfil.	**Operativo**
POST	/api/logout	Invalida la sesión actual.	**Operativo**
GET	/api/reportes/hospitales/resumen	Reporte 3: resumen por hospital.	**Implementado**
GET	/api/reportes/hospitales/mayores-de-cien	Reporte 4: hospitales con más de 100 pacientes (top 5).	**Implementado**
GET	/api/reportes/proceso	Reporte 7: resumen del proceso de atención.	**Implementado**
GET	/api/reportes/medicos	Reporte 2: listado de médicos.	**Implementado**
GET	/api/reportes/pacientes ?area=&codigo=	Reporte 1: listado de pacientes por área y código.	**Implementado**
GET	/api/reportes/pacientes/no-atendidos ?area=&codigo=	Reporte 6: pacientes no atendidos.	**Implementado**
GET	/api/reportes/consultas/informe ?area=&codigo=	Reporte 5: informe durante las consultas.	**Implementado**
GET	/api/reportes/consultas/exitosas ?area=&codigo=	Reporte 9: consultas exitosas.	**Implementado**
GET	/api/reportes/unidades/turnos-por-revisar	Reporte 8: unidades que deben revisar turnos.	**Implementado**

Los servicios CRUD del dominio existen, pero todavía no están expuestos mediante controladores REST. Por eso el panel puede consultar reportes y autenticarse, pero aún no puede crear o modificar hospitales, médicos, pacientes o turnos contra la base de datos.

7.3 Autenticación y seguridad

- **Contraseñas:** se almacenan con hash BCrypt; el login compara la contraseña enviada con el hash almacenado.
- **Sesión:** se usa HttpSession con cookie JSESSIONID; al iniciar sesión se cambia el identificador de sesión y se guardan username, email y rol.
- **Autorización:** el panel aplica visibilidad por rol; los módulos Profesionales y Reportes están reservados a ADMIN.
- **CORS:** la API acepta orígenes localhost y 127.0.0.1, con credenciales habilitadas.
- **CSRF:** está deshabilitado para simplificar el consumo desde el frontend.
- **Manejo de errores:** las respuestas de error mantienen el formato { ok: false, error: "..." } para que el frontend pueda mostrarlas.

7.4 Observación sobre la configuración de datos

La aplicación está configurada para conectarse a la base Login, donde viven los usuarios. Sin embargo, el dominio hospitalario (hospitales, pacientes, turnos, reportes) está en la base Gestion Hospitalaria. Al no existir una única base con ambas partes, los endpoints de reportes fallan al ejecutarse. Esta es la brecha técnica más importante a resolver antes de la entrega: unificar ambas bases o configurar dos fuentes de datos.

8. Frontend y experiencia de usuario

8.1 Páginas y navegación

El frontend tiene dos páginas. index.html funciona como portada y centro de acceso: presenta el sistema, permite iniciar sesión y ofrece un registro en dos pasos (datos personales y credenciales). dashboard.html es el panel principal, con barra lateral, encabezado, menú de cuenta y navegación por hash entre secciones, sin recargar la página.

El panel incluye tres modales de trabajo: crear turno, registrar paciente y editar perfil. También incorpora notificaciones toast, filtros por especialidad y estado, búsqueda de pacientes y turnos, y botones de exportación que hoy muestran un aviso de preparación.

8.2 Módulos del panel

Sección	Contenido implementado	Estado
Resumen	Saludo personalizado, fecha, tarjetas de indicadores, agenda del día, cola de atención y pacientes recientes.	**Implementado**
Turnos	Indicadores de turnos, agenda, resumen por especialidad, filtros, tabla de turnos y acciones de detalle.	**Implementado**
Pacientes	Directorio, búsqueda, filtros por estado, resumen por especialidad y actividad reciente.	**Implementado**
Profesionales	Vista informativa de módulo reservado a administradores.	**Pendiente**
Reportes	Vista informativa de módulo reservado a administradores.	**Pendiente**
Configuración	Vista informativa de preferencias del sistema.	**Pendiente**
Ayuda	Vista informativa de centro de ayuda.	**Pendiente**

8.3 Comunicación con el backend

- **Detección de la API:** api.js prueba primero el mismo origen si la página se sirve en el puerto 8080 y, si no, el host actual en ese puerto; así funciona tanto con Spring Boot como con un servidor estático.
- **Sesión:** todas las peticiones incluyen credentials: include para enviar la cookie de sesión.
- **Normalización:** las respuestas se validan por content-type y por la presencia del campo ok, lo que evita aceptar respuestas HTML de un servidor equivocado.
- **Respaldo visual:** si el backend no responde, el dashboard usa datos de demostración para no bloquear la revisión del diseño; un indicador avisa que se está en modo demo.

8.4 Diseño y experiencia

La interfaz tiene una identidad visual propia: paleta azul petróleo y verde azulado, tipografías Manrope y DM Sans, tarjetas con bordes suaves, chips de estado, avatares y una barra lateral con iconos. El diseño es responsive, con menú móvil y adaptación de tablas y tarjetas. La iconografía combina Bootstrap Icons y Lucide. Los estilos están separados entre el login (index.css) y el panel (dashboard.css), aunque el repositorio conserva copias de respaldo con nombres como dash-prueba.css y prueba.css que conviene depurar.

9. Estado de avance

El avance se evalúa por áreas y no solo por pantallas. La siguiente tabla resume qué está terminado, qué está en progreso y qué falta, con la evidencia correspondiente.

Área	Estado	Evidencia y detalle
Modelo de datos y lógica SQL	**Completado**	12 tablas, 33 funciones en el dominio, 2 disparadores y datos de prueba cargados.
Autenticación y perfiles	**Completado**	Login, registro en dos pasos, logout, /me, edición de perfil, BCrypt y roles ADMIN/MEDICO.
Panel de control (UI)	**Completado**	Dashboard responsive con Resumen, Turnos y Pacientes; modales, filtros, búsqueda y navegación.
API de reportes	**Completado**	9 endpoints REST y servicios JDBC que invocan funciones PL/pgSQL.
Integración panel ↔️ datos reales	**En progreso**	El panel consume el resumen de hospitales y usa datos demo como respaldo; turnos y consultas aún no persisten desde la UI.
CRUD REST del dominio	**En progreso**	Los 8 servicios JDBC existen; faltan los controladores REST que los expongan.
CRUD en el panel	**Pendiente**	Profesionales, estructura hospitalaria y gestión de usuarios aún no tienen pantalla funcional.
Módulo de reportes en el panel	**Pendiente**	La vista es un placeholder; los endpoints ya están disponibles.
Seguridad de la API	**En progreso**	BCrypt y sesión operativos; endpoints abiertos, CSRF deshabilitado, CORS limitado a localhost y credenciales en configuración.
Versionado de la base de datos	**Pendiente**	No existe la carpeta database/ con scripts de esquema, datos y funciones.
Despliegue reproducible	**En progreso**	start.sh funciona y el servicio systemd está configurado; docker-compose está incompleto y no hay Dockerfile.
Pruebas automatizadas	**Pendiente**	No hay código en src/test ni pruebas de integración.
Documentación	**En progreso**	README mínimo y estructura.txt aspiracional; este documento cubre el estado actual.

9.1 Evidencia cuantitativa

Indicador	Valor
Clases Java del backend	35 archivos, ~3.600 líneas
Código del frontend	~8.300 líneas entre HTML, CSS y JavaScript
Tablas de base de datos	12 (9 de dominio + 3 de usuarios)
Funciones PL/pgSQL	33 en el dominio + 8 en la base de usuarios
Disparadores	2 disparadores sobre 5 eventos
Reportes definidos	9
Endpoints REST	14 (5 de autenticación + 9 de reportes)
Datos de prueba	596 pacientes, 26 turnos, 47 informes, 8 hospitales
Usuarios de prueba	2 (1 ADMIN y 1 MEDICO)
Scripts de operación	start.sh, stop.sh y servicio systemd

10. Despliegue y ejecución local

10.1 Requisitos

- **Java:** JDK 17 o superior; el servicio systemd actual usa JDK 25, por lo que conviene verificar la compatibilidad con Spring Boot 3.2.5.
- **Maven:** 3.8 o superior para compilar y ejecutar con spring-boot:run.
- **PostgreSQL:** 17 en localhost:5432, con las bases y credenciales configuradas en application.properties.
- **Navegador:** cualquier navegador moderno con soporte para módulos ES.

10.2 Arranque con scripts

El script start.sh exporta la ubicación del frontend como recursos estáticos, arranca Spring Boot con Maven en segundo plano, guarda el PID y el log en .run/, espera a que el puerto esté disponible y abre el navegador. stop.sh detiene el grupo de procesos de forma ordenada.

```
./start.sh              # arranca y abre http://localhost:8080
NO_OPEN=1 ./start.sh    # arranca sin abrir el navegador
./stop.sh               # detiene el servidor
```

Spring Boot sirve el frontend y la API en el mismo puerto, por lo que la aplicación queda disponible en http://localhost:8080. Para acceder desde otro dispositivo de la red local se usa http://IP-DEL-EQUIPO:8080, siempre que el firewall permita el puerto 8080.

10.3 Servicio systemd

El archivo deploy/proyecto-dw.service permite ejecutar el proyecto como servicio de usuario. Define el directorio de trabajo, las variables NO_OPEN y JAVA_HOME, y usa start.sh/stop.sh como comandos de arranque y parada, con un timeout de 180 segundos.

10.4 Docker Compose

El repositorio incluye un docker-compose.yml con dos servicios: PostgreSQL 17 y el backend. La configuración es parcial: el backend declara build: . pero no existe un Dockerfile en la raíz; el healthcheck usa nombres de usuario y base de ejemplo; y el volumen pgdata es un volumen nombrado de Docker, no una carpeta del proyecto. Por lo tanto, el despliegue con contenedores todavía no es reproducible sin ajustes.

10.5 Portabilidad de la base de datos

Para que el proyecto se pueda ejecutar en otra PC sin depender de la instalación local, conviene versionar la base de datos en una carpeta database/ dentro del repositorio. La forma recomendada es guardar scripts SQL (esquema, datos y funciones) y un script de restauración, no copiar los archivos internos de PostgreSQL, porque dependen de la versión, del sistema operativo y del usuario que ejecuta el servidor.

> **Recomendación para la carpeta database/**
>
> Crear database/01_schema.sql, database/02_seed.sql y database/03_functions.sql, más un restore.sh que cree las bases y ejecute los scripts. Alternativamente, usar Docker con un bind mount ./database/pgdata:/var/lib/postgresql/data para que los datos queden dentro del proyecto. En ambos casos, agregar database/pgdata al .gitignore para no versionar datos binarios.

11. Observaciones técnicas y riesgos

Las siguientes observaciones surgen de la revisión del código y de la base de datos. No impiden demostrar el proyecto, pero sí condicionan la entrega y la puesta en marcha en otro equipo.

#	Hallazgo	Impacto	Recomendación
1	La aplicación apunta a la base Login, pero el dominio hospitalario está en Gestion Hospitalaria.	Los reportes y el CRUD del dominio fallan al ejecutarse.	Unificar ambas bases o configurar dos fuentes de datos y separar los repositorios.
2	No hay scripts SQL versionados en el repositorio.	La base solo existe en la instancia local; no se puede reconstruir en otra PC.	Crear database/ con esquema, datos, funciones y restore.sh.
3	Los servicios CRUD no están expuestos por REST.	El panel no puede crear ni modificar hospitales, médicos, pacientes o turnos.	Agregar controladores REST y conectar los formularios del panel.
4	El dashboard usa datos demo cuando el backend falla.	La interfaz puede parecer funcional sin estarlo; los cambios no persisten.	Mostrar un aviso claro de modo demo y priorizar la integración real.
5	Los endpoints /api/** están abiertos y el CSRF está deshabilitado.	Cualquier cliente puede invocar la API; solo /me valida sesión.	Aplicar autorización por rol, proteger los endpoints y evaluar CSRF.
6	Credenciales de base de datos en application.properties.	Riesgo de exposición al compartir el repositorio.	Externalizar a variables de entorno o a un archivo .env no versionado.

#	Hallazgo	Impacto	Recomendación
7	CORS limitado a localhost y 127.0.0.1.	El acceso desde otro dispositivo de la red puede quedar bloqueado si se usa otro origen.	Agregar el host/IP de la red local o servir siempre desde el mismo origen.
8	docker-compose.yml incompleto y sin Dockerfile.	El despliegue con contenedores no funciona tal como está.	Crear el Dockerfile del backend y corregir healthcheck, build y volumen.
9	El servicio systemd usa JDK 25 y el proyecto declara Java 17.	Posible incompatibilidad no verificada con Spring Boot 3.2.5.	Alinear JAVA_HOME a un JDK 17 o 21 y probar el arranque.
10	Hay artefactos generados y archivos duplicados en el repositorio (target/, bin/, CSS de respaldo).	Ruido en el historial y riesgo de confusiones.	Limpiar el repositorio, ampliar .gitignore y eliminar copias innecesarias.
11	No hay pruebas automatizadas.	Las regresiones se detectan solo manualmente.	Agregar pruebas unitarias de servicios y de integración de endpoints.
12	estructura.txt describe una estructura objetivo distinta de la real.	Puede confundir a quien retome el proyecto.	Actualizar la documentación o moverla a una sección de roadmap.

12. Próximos pasos

El roadmap se ordena por impacto en la entrega: primero se asegura que el sistema funcione con datos reales y sea reproducible; después se completan los módulos de gestión y reportes; por último se refuerzan seguridad, pruebas y despliegue.

Prioridad	Acción	Resultado esperado
Alta	Unificar la base de datos o configurar las dos fuentes.	Reportes y CRUD funcionando contra los datos reales.
Alta	Crear database/ con esquema, datos, funciones y restore.sh.	Instalación reproducible en cualquier PC con PostgreSQL.
Alta	Exponer los servicios CRUD mediante controladores REST.	API completa para hospitales, departamentos, unidades, médicos, pacientes, turnos, consultas e informes.
Alta	Conectar los formularios del panel a la API.	Alta y edición de turnos, pacientes y perfiles con persistencia real.
Media	Construir la vista de Reportes con los 9 endpoints.	Módulo de analítica utilizable por administradores.
Media	Implementar el módulo Profesionales.	Gestión de médicos, especialidades y unidades desde el panel.
Media	Reforzar seguridad: autorización por rol, secretos externos, CORS y sesión.	API protegida y configuración apta para otros entornos.
Media	Implementar exportación real (CSV/PDF) de listados.	Reportes descargables y compartibles.
Baja	Crear Dockerfile y corregir docker-compose.	Despliegue con contenedores en un comando.

Prioridad	Acción	Resultado esperado
Baja	Agregar pruebas automatizadas y CI.	Mayor confianza en cada cambio.
Baja	Actualizar documentación y limpiar el repositorio.	Proyecto fácil de retomar y presentar.

13. Conclusiones

El proyecto ya cuenta con una base técnica sólida y coherente. El modelo de datos hospitalario es completo, las reglas de negocio críticas están implementadas en la base de datos y la interfaz presenta un nivel de diseño y usabilidad que permite demostrar el producto. La autenticación con roles, el panel responsive y los nueve reportes definen con claridad el alcance funcional.

El principal desafío no es rehacer, sino integrar: unificar la configuración de la base de datos, versionarla en el repositorio, exponer los CRUD por REST y conectar el panel a los datos reales. Con esas acciones, el proyecto pasará de un MVP visual a una aplicación completa y reproducible, lista para ejecutarse en cualquier PC y para ser presentada como solución de gestión hospitalaria.

En términos de aprendizaje y arquitectura, el proyecto demuestra el uso combinado de Spring Boot, PostgreSQL, JPA y JDBC, seguridad con sesiones y BCrypt, y un frontend moderno sin framework. Esa combinación constituye una base extensible para futuras funcionalidades como facturación, farmacia, notificaciones o integración con otros sistemas.

Anexo A. Funciones de base de datos

A.1 Dominio hospitalario (base Gestion Hospitalaria)

Grupo	Funciones
Altas	insertarHospital, insertarDepartamento, insertarUnidad, insertarMedico, insertarPaciente, insertarTurno, insertarConsulta
Modificaciones	modificarHospital, modificarDepartamento, modificarUnidad, modificarMedico, modificarPaciente, modificarTurno, modificarConsulta
Bajas	eliminarHospital, eliminarDepartamento, eliminarUnidad, eliminarMedico, eliminarPaciente, eliminarTurno, eliminarConsulta, eliminarInforme
Reportes	resumenHospitales, listadoMasDeCien, resumenProceso, listadoMedicos, listadoPacientes, pacientesNoAtendidos, informeDuranteConsultas, consultasExitosas, revisarTurnos
Disparadores	actualizar_informe_trigger (AFTER INSERT/UPDATE/DELETE en Consulta), limitarmedicosporunidad (BEFORE INSERT/UPDATE en Medico)

A.2 Usuarios (base Login)

Función	Propósito
func_crear_usuario	Crea un usuario y su perfil.
func_login	Valida credenciales de acceso.
func_obtener_usuario	Devuelve los datos del usuario.
func_modificar_datos_usuario	Actualiza el perfil.
func_modificar_contrasena	Cambia la contraseña (dos variantes).
func_actualizar_contrasena_por_email	Recuperación de contraseña por correo.
func_eliminar_usuario	Elimina un usuario.

Anexo B. Glosario

Término	Significado
API REST	Interfaz HTTP que expone operaciones sobre recursos mediante métodos GET, POST, PUT y DELETE.
BCrypt	Algoritmo de hash adaptativo usado para almacenar contraseñas de forma segura.
CRUD	Conjunto de operaciones de alta, consulta, modificación y baja sobre una entidad.
JPA / Hibernate	Estándar y motor de persistencia objeto-relacional para mapear clases Java a tablas.
JDBC	API de Java para ejecutar SQL directamente contra una base de datos.
PL/pgSQL	Lenguaje procedural de PostgreSQL para escribir funciones y disparadores.
Trigger	Disparador que ejecuta lógica automáticamente ante un evento de la base de datos.

Término	Significado
Sesión HTTP	Mecanismo que mantiene el estado de autenticación entre peticiones mediante una cookie.
MVP	Producto mínimo viable: versión con las funciones esenciales para validar el proyecto.

Fin del documento.